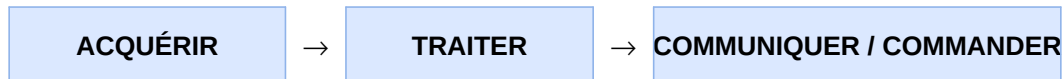


Cours & Synthèse SIN : Acquérir, Traiter, Communiquer

Notions indispensables mises en jeu dans le TP « Aide au stationnement Grove ».

1 — La chaîne d'information

La chaîne d'information est la partie du système qui capte les données, les analyse et donne des ordres. Elle se compose de trois fonctions principales :



Acquérir — Recevoir une information provenant de l'environnement ou de l'utilisateur. *Exemples : capteur de distance ultrasons, bouton-poussoir.*

Traiter — Exécuter un programme pour décider de l'action à réaliser. *Exemple : microcontrôleur (Arduino Uno R3).*

Communiquer / Commander — Restituer une information à l'utilisateur ou envoyer un ordre à un actionneur. *Exemples : LED, buzzer piézoélectrique.*

■ **À retenir** : un bouton est bien une entrée (IN) : il transmet au système l'état choisi par l'utilisateur. Une LED ou un buzzer sont des sorties (OUT).

2 — Deux formes d'informations utiles

Type de signal	Exemples	Usage dans le programme
Valeur numérique	27 cm, 21.4 °C, 612 lux	Comparer à un seuil, faire un calcul, afficher.
État logique / TOR	Appuyé / Relâché, Ouvert / Fermé	Déclencher ou interdire instantanément une action.

TOR signifie **Tout Ou Rien** (signal binaire).

Dans un programme Arduino, un état logique prend la valeur **HIGH** (5V / Vrai) ou **LOW** (0V / Faux).

3 — Lire une condition (structure conditionnelle)

```
if (distance <= 30)
{
  digitalWrite(LED, HIGH); // Allume la LED si distance <= 30 cm
}
```

if signifie « si ». Le programme évalue la condition entre parenthèses. Il exécute le bloc entre accolades { } uniquement si cette condition est vraie.

Les opérateurs de comparaison indispensables :

- < : inférieur à
- <= : inférieur ou égal à
- > : supérieur à
- >= : supérieur ou égal à
- == : égal à (attention : double == pour comparer)

- != : différent de

4 — Gérer plusieurs zones avec if / else if / else

Lorsqu'un système possède plusieurs zones de fonctionnement (comme les bips d'un radar de recul), il faut tester les conditions dans un ordre cohérent (souvent du plus restrictif au plus général).

```
if (distance <= 15)
{
// Zone 1 : Danger immédiat (ex : bip continu, LED rouge)
}
else if (distance <= 30)
{
// Zone 2 : Zone intermédiaire (ex : bip lent, LED orange)
}
else
{
// Zone 3 : Zone éloignée / Sécurité (ex : pas de son, LED verte)
}
```

■ **Règle de syntaxe stricte** : dans cette structure, dès qu'une condition est vraie, les blocs suivants sont ignorés.

Interdiction mathématique : en C/C++, on n'écrit jamais `15 < distance <= 30`. On utilise des opérateurs logiques si nécessaire, ou l'imbrication `else if` ci-dessus.

5 — Commander une sortie (actionneurs)

Pour modifier l'état des composants de sortie (LED et buzzer Grove), on utilise des fonctions spécifiques :

■ Gérer une LED

- `digitalWrite(LED, HIGH);` → Allumer la LED (envoi du 5V).
- `digitalWrite(LED, LOW);` → Éteindre la LED (envoi du 0V).

■ Gérer un buzzer

- `tone(BUZZER, 1000);` → Produire un son (ici à une fréquence de 1000 Hz).
- `noTone(BUZZER);` → Arrêter immédiatement le son du buzzer.